

ASSESSMENT TOOL 1

GIT HUB LINK: https://github.com/Najeeb2006/compiler-design/tree/main/CO2_AT1

COURSE CODE: CSA1442

COURSE NAME: Compiler Design

AL NAJEEB A
192424325

CO2 AT1 CSA1402 Alnajeeb.A

Q1. Predictive Parser Design (Concept mapping) 192424325

ZONE 1: The grammar

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow TE' | \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow * FT' | \epsilon \\ F &\rightarrow (E) | id \end{aligned}$$

The grammar is left recursive, and properly left factored

Zone 2:

First Sets

$$\begin{aligned} \text{First}(E) &= \{ (, id \} \\ \text{First}(E') &= \{ +, \epsilon \} \\ \text{First}(T) &= \{ (, id \} \\ \text{First}(T') &= \{ *, \epsilon \} \\ \text{First}(F) &= \{ (, \$ \} \end{aligned}$$

FOLLOW SETS

$$\begin{aligned} \text{FOLLOW}(E) &= \{), \$ \} \\ \text{FOLLOW}(E') &= \{), \$ \} \\ \text{FOLLOW}(T) &= \{ +,), \$ \} \\ \text{FOLLOW}(T') &= \{ +,), \$ \} \\ \text{FOLLOW}(F) &= \{ *, +,), \$ \} \end{aligned}$$

Zone 3:

Non Terminals

ID + * () \$

E

$E \rightarrow TE'$

$E \rightarrow TE'$

E'

$E \rightarrow TE'$

$T \rightarrow FT'$

$E \rightarrow +TE'$

T

$T \rightarrow FT'$

T'

$T \rightarrow FT'$

$T' \rightarrow \epsilon$

F

$F \rightarrow id$

$F \rightarrow (t)$

Zone 4:

Stack

I.B

P. Action

\$ E

id + id * id \$

$E \rightarrow TE'$

\$ E' T

id + id * id \$

$T \rightarrow FT'$

\$ E' T' F

id + id * id \$

$F \rightarrow id$

\$ E' T' id

\$ E' T'

\$ E'

\$ E' T' +

\$ E' T' F

\$ E' T' id

\$ E' T'

\$ E' T' F *

\$ E' T' F

\$ E' T' id

\$ E' T'

\$ E'

id + id * id \$

+ id * id \$

+ id * id \$

id + id \$

id * id \$

id + id \$

* id \$

* id \$

id \$

id \$

\$

\$

match id

$T' \rightarrow \epsilon$

$E' \rightarrow T E'$

Match +

$T \rightarrow F T'$

match id

$T' \rightarrow * F T'$

match *

$F \rightarrow id$

match id

$T' \rightarrow \epsilon$

$E' \rightarrow \epsilon$

Snippet OF

PREDICTIV PARSER

Class Predictive parser :

```
def __init__(self, start):
```

```
    self.elements = [start]
```

```
def push(self, production):
```

```
    self.elements.extend(reversed(prod))
```

```
def match(self, current_token):
```

```
    if self.elements[-1] == current_token:
```

```
        return self.elements.pop()
```

Q2. Shift-Reduce Parsing (Application Level)

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #define MAX 100
6
7 char stack[MAX][20];
8 int top = -1;
9
10 void push(char *sym) {
11     top++;
12     strcpy(stack[top], sym);
13 }
14
15 void pop() {
16     if (top >= 0) top--;
17 }
18
19 void printStack() {
20     printf("Stack: [ ");
21     for (int i = 0; i <= top; i++)
22         printf("%s ", stack[i]);
23     printf("]");
24 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS CODE REFERENCE LOG QUERY RESULTS

PS C:\Users\ABDUL AZEEZ\Downloads\CompilerDesign_Q2_Q5_Q8> .\Q2.exe

Step	Stack	Input	Action
1	Stack: [\$]	Input: [id + id * id \$]	Shift id
2	Stack: [\$ id]	Input: [+ id * id \$]	Reduce E -> id
3	Stack: [\$ E]	Input: [+ id * id \$]	Shift +
4	Stack: [\$ E +]	Input: [id * id \$]	Shift id
5	Stack: [\$ E + id]	Input: [* id \$]	Reduce E -> id
6	Stack: [\$ E + E]	Input: [* id \$]	Shift *
7	Stack: [\$ E + E *]	Input: [id \$]	Shift id
8	Stack: [\$ E + E * id]	Input: [\$]	Reduce E -> id
9	Stack: [\$ E + E * E]	Input: [\$]	Reduce E -> E * E
10	Stack: [\$ E + E]	Input: [\$]	Reduce E -> E + E
11	Stack: [\$ E]	Input: [\$]	ACCEPT

Q 5. Advanced LL(1) Parser Evaluation and Design

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5
6 #define MAX 100
7 #define EPSILON "eps"
8
9 char *NT[] = {"S", "S'", "T", "T'", "F"};
10 char *TM[] = {"id", "+", "*", "(", ")", "$"};
11
12
13 char *table[5][6] = {
14     /*      id      +      *      (      )      $ */
15     /* S */ {"TS'", "", "", "TS'", "", ""},
16     /* S' */ {"", "+TS'", "", "", EPSILON, EPSILON},
17     /* T */ {"FT'", "", "", "FT'", "", ""},
18     /* T' */ {"", EPSILON, "+FT'", "", EPSILON, EPSILON},
19     /* F */ {"id", "", "", "(S)", "", ""},
20 };
21
22 char parseStack[MAX][10];
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS CODE REFERENCE LOG QUERY RESULTS

PS C:\Users\ABDUL AZEEZ\Downloads\CompilerDesign_Q2_Q5_Q8> .\Q5.exe

1	Stack: [\$ \$]	Input: [id + id * id \$]	Output S -> TS'
2	Stack: [\$ \$ T]	Input: [id + id * id \$]	Output T -> FT'
3	Stack: [\$ \$ T' F]	Input: [id + id * id \$]	Output F -> id
4	Stack: [\$ \$ S' T' id]	Input: [id + id * id \$]	Match id
5	Stack: [\$ \$ T']	Input: [+ id * id \$]	Output T' -> eps
6	Stack: [\$ \$ S']	Input: [+ id * id \$]	Output S' -> +TS'
7	Stack: [\$ \$ S' T +]	Input: [+ id * id \$]	Match +
8	Stack: [\$ \$ S' T]	Input: [id * id \$]	Output T -> FT'
9	Stack: [\$ \$ T' F]	Input: [id * id \$]	Output F -> id
10	Stack: [\$ \$ S' T' id]	Input: [id * id \$]	Match id
11	Stack: [\$ \$ T']	Input: [* id \$]	Output T' -> *FT'
12	Stack: [\$ \$ T' F *]	Input: [* id \$]	Match *
13	Stack: [\$ \$ S' T' F]	Input: [id \$]	Output F -> id
14	Stack: [\$ \$ S' T' id]	Input: [id \$]	Match id
15	Stack: [\$ \$ S']	Input: [\$]	Output T' -> eps
16	Stack: [\$ \$ S']	Input: [\$]	Output S' -> eps
17	Stack: [\$ \$]	Input: [\$]	ACCEPT

Q8. Ambiguity Removal and Parsing Strategy Evaluation

